

• Medium → #16, 3 Sum Closest

▲ Problem Statement → Given an integer array "nums" & an integer "target", find three distinct elements whose sum is closest to "target".

Return the sum of those three integers.

▲ Main Problem Solution → Sort the array & use the "Two Pointer" approach.

Fix one element "nums[i]", then use "left" & "right" pointers to find the closest possible sum.

Compare the current sum with "target" using the difference :-

$$\text{"abs(target - sum)"}$$

If the current sum is closer, update "closest".

▲ Brute Force Method - Use three nested loops to check every possible triplet & keep track of the closest sum.

Time $\rightarrow O(n^3)$

Space $\rightarrow O(1)$

▲ Algorithm $\rightarrow$ 1. Sort the array

2. Initialize "closest" with the sum of the first three elements.

3. Loop through the array using "i".

4. Set "left = i + 1" & "right = n - 1".

5. Calculate "sum = nums[i] + nums[left] + nums[right]"

6. If "sum" is closer to "target", update "closest"

7. If "sum < target", move left forward.

8. If "sum > target", move right backward.

9 | If " $sum == target$ ", return it immediately.

10 | Return "closest"

▲ Important Points → • Sort the array first.

• Fix one element & use two pointers for the remaining two.

• " $sum < target$ " → move "left"

• " $sum > target$ " → move "right"

• Use " $abs(target - sum)$ " to check which sum is closer.

• If " $sum == target$ ", that's the closests possible answer, so return immediately.

▲ Complexity → Time → $O(n^2)$

Space → $O(1)$ excluding the sorting implementation.